Fractional Knapsack using Greedy Method

```
n = int(input("Enter number of items: "))
items = []
for i in range(n):
    weight = float(input("Enter weight of item: "))
    profit = float(input("Enter profit of item: "))
    ratio = profit / weight
    items.append((weight, profit, ratio))
capacity = float(input("Enter knapsack capacity: "))
# Sort items by profit/weight ratio
items.sort(key=lambda x: x[2], reverse=True)
total_profit = 0
for weight, profit, ratio in items:
    if capacity == 0:
        break

    if weight <= capacity:
        total_profit += profit
        capacity -= weight
    else:
        fraction = capacity / weight
        total_profit += profit * fraction
        capacity = 0
print("Maximum Profit:", total_profit)
```

#if in question they give table:

```
def solve_knapsack_greedy():
    # Input Data
    items = ["A", "B", "C", "D"]
    weights = [8, 3, 2, 10]
    profits = [40, 30, 20, 50]
```

```
capacity = 12

# 1. Calculate profit/weight ratios and combine item details
item_details = []
for i in range(len(items)):
  ratio = profits[i] / weights[i]   #Calculates the value density.
  item_details.append({
    "name": items[i],
    "weight": weights[i],
    "profit": profits[i],
    "ratio": ratio
  })

# 2. Sort items by ratio in descending order
item_details.sort(key=lambda x: x["ratio"], reverse=True)

# Print the item ordering to show the greedy process
print("Greedy Ordering (Highest Profit/Weight Ratio first):")
print(f"{'Item':<6}{'Profit':<8}{'Weight':<8}{'Ratio':<6}")
for item in item_details:
  print(f"{item['name']:<6}{item['profit']:<8}{item['weight']:<8}{item['ratio']:<6.2f}")
print("-" * 35)

# 3. Pack the knapsack greedily
total_profit = 0.0
remaining_capacity = capacity
selected_items = []

for item in item_details:
  if remaining_capacity == 0:
    break
```

```
    if item["weight"] <= remaining_capacity:
        # Take the full item
        total_profit += item["profit"]
        remaining_capacity -= item["weight"]
        selected_items.append(f"{item['name']} (100%)")
    else:
        # Take a fraction of the item to fill the remaining capacity
        fraction = remaining_capacity / item["weight"]
        fraction_profit = item["profit"] * fraction
        total_profit += fraction_profit
        selected_items.append(f"{item['name']} ({fraction * 100:.1f}%)")
        remaining_capacity = 0  # Knapsack is completely full

# 4. Output results
print(f"\nMaximum Profit Obtained: {total_profit:.2f}")
print(f"Selected Items: {', '.join(selected_items)}")

solve_knapsack_greedy()
```

1. **Items B and C** both have the highest density ratio of **10.0**. The algorithm takes all of item B (3kg) and all of item C (2kg).
    - Used capacity = $3 + 2 = 5\text{kg}$.
    - Remaining capacity = $12 - 5 = 7\text{kg}$.
    - Accumulated profit = $30 + 20 = 50$.
2. **Item A** is looked at next (ratio of 5.0). The knapsack only has 7kg left, but Item A weighs 8kg.
3. The greedy approach takes a **fraction** of A: $\frac{7}{8} = 87.5\%$.
    - Profit from fraction = $40 \times \frac{7}{8} = 35$.
4. **Total Profit** = $50 + 35 = \mathbf{85.00}$
5. `.sort()`: A built-in Python method that rearranges elements in a list.

6. **`reverse=True`**: Tells Python to sort from **highest to lowest** (descending order).
7. **`key=lambda x: x["ratio"]`**:
    - **What is `lambda`?** A `lambda` function is a small, one-line, anonymous function (a function without a name). It acts as a shortcut. Instead of writing a full `def get_ratio(x): return x["ratio"]` block elsewhere, you can write it instantly inline.
    - **How it works here**: The sorting tool looks at each item dictionary (represented by `x`) and extracts just the value under the `"ratio"` key. Python then uses those ratio values to decide the sorting order.
8. Python uses a highly optimized sorting algorithm called Timsort. Sorting a list of N elements based on their ratios takes **(O(N log N)**time. **This is the most time-consuming step in the entire program. {Total Time}=O(N)+O(Nlog N)+O(N))Because (O(N log N)) grows much faster than O(N), the smaller terms are dropped. Therefore, the final overall time complexity is O(N \log N).**
9. he space complexity is **O(N)** because the code creates a new list (`item_details`) to store the dictionary details and ratios for the N items.
10.